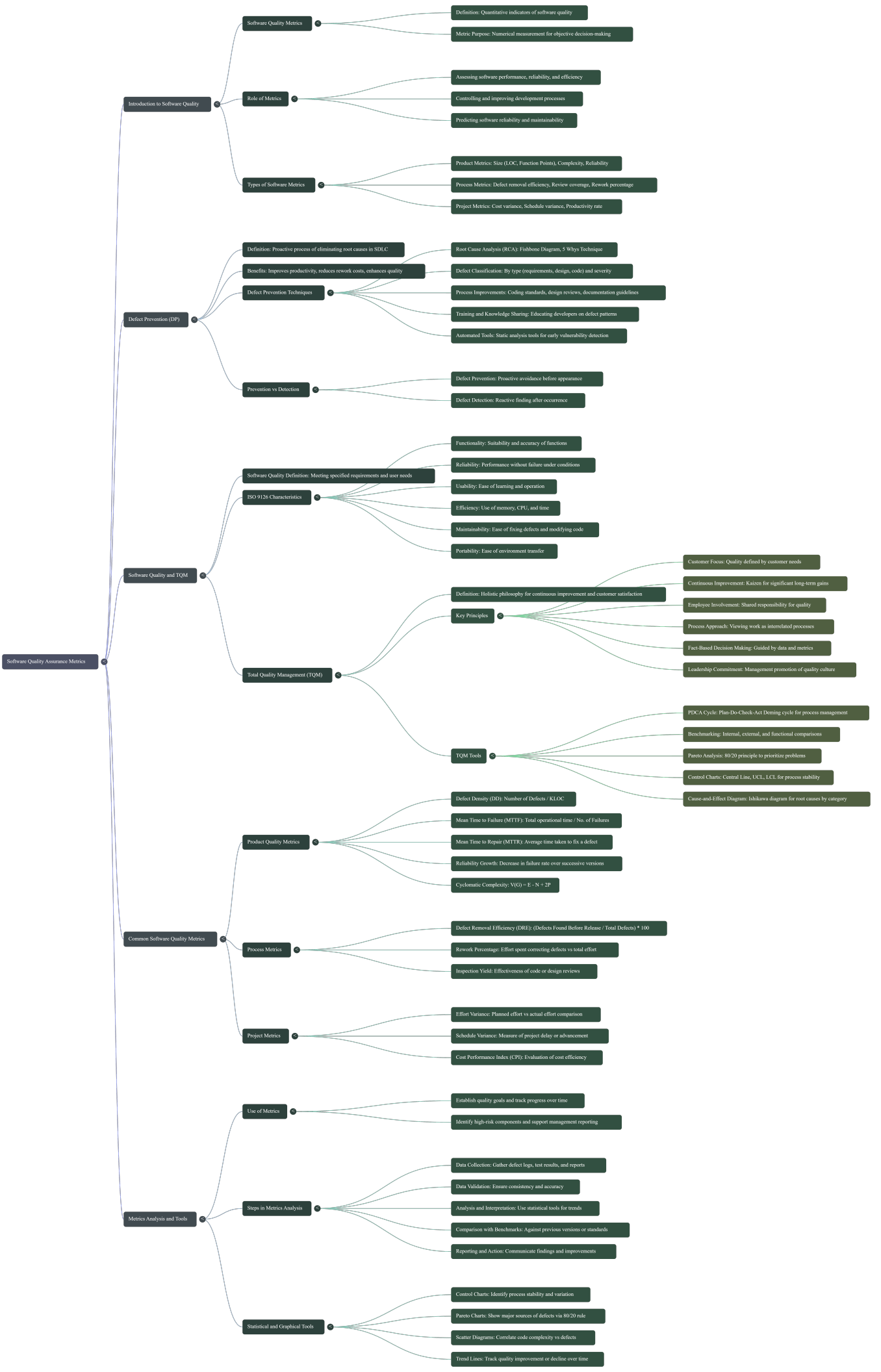

Software Quality Assurance Metrics
Introduction to Software Quality
Software Quality Metrics
Definition: Quantitative indicators of software quality
Metric Purpose: Numerical measurement for objective decision-making
Role of Metrics
Assessing software performance, reliability, and efficiency
Controlling and improving development processes
Predicting software reliability and maintainability
Types of Software Metrics
Product Metrics: Size (LOC, Function Points), Complexity, Reliability
Process Metrics: Defect removal efficiency, Review coverage, Rework percentage
Project Metrics: Cost variance, Schedule variance, Productivity rate
Defect Prevention (DP)
Definition: Proactive process of eliminating root causes in SDLC
Benefits: Improves productivity, reduces rework costs, enhances quality
Defect Prevention Techniques
Root Cause Analysis (RCA): Fishbone Diagram, 5 Whys Technique
Defect Classification: By type (requirements, design, code) and severity
Process Improvements: Coding standards, design reviews, documentation guidelines
Training and Knowledge Sharing: Educating developers on defect patterns
Automated Tools: Static analysis tools for early vulnerability detection
Prevention vs Detection
Defect Prevention: Proactive avoidance before appearance
Defect Detection: Reactive finding after occurrence
Software Quality and TQM
Software Quality Definition: Meeting specified requirements and user needs
ISO 9126 Characteristics
Functionality: Suitability and accuracy of functions
Reliability: Performance without failure under conditions
Usability: Ease of learning and operation
Efficiency: Use of memory, CPU, and time
Maintainability: Ease of fixing defects and modifying code
Portability: Ease of environment transfer
Total Quality Management (TQM)
Definition: Holistic philosophy for continuous improvement and customer satisfaction
Key Principles
Customer Focus: Quality defined by customer needs
Continuous Improvement: Kaizen for significant long-term gains
Employee Involvement: Shared responsibility for quality
Process Approach: Viewing work as interrelated processes
Fact-Based Decision Making: Guided by data and metrics
Leadership Commitment: Management promotion of quality culture
TQM Tools
PDCA Cycle: Plan-Do-Check-Act Deming cycle for process management
Benchmarking: Internal, external, and functional comparisons
Pareto Analysis: 80/20 principle to prioritize problems
Control Charts: Central Line, UCL, LCL for process stability
Cause-and-Effect Diagram: Ishikawa diagram for root causes by category
Common Software Quality Metrics
Product Quality Metrics
Defect Density (DD): Number of Defects / KLOC
Mean Time to Failure (MTTF): Total operational time / No. of Failures
Mean Time to Repair (MTTR): Average time taken to fix a defect
Reliability Growth: Decrease in failure rate over successive versions
Cyclomatic Complexity: V(G) = E - N + 2P
Process Metrics
Defect Removal Efficiency (DRE): (Defects Found Before Release / Total Defects) * 100
Rework Percentage: Effort spent correcting defects vs total effort
Inspection Yield: Effectiveness of code or design reviews
Project Metrics
Effort Variance: Planned effort vs actual effort comparison
Schedule Variance: Measure of project delay or advancement
Cost Performance Index (CPI): Evaluation of cost efficiency
Metrics Analysis and Tools
Use of Metrics
Establish quality goals and track progress over time
Identify high-risk components and support management reporting
Steps in Metrics Analysis
Data Collection: Gather defect logs, test results, and reports
Data Validation: Ensure consistency and accuracy
Analysis and Interpretation: Use statistical tools for trends
Comparison with Benchmarks: Against previous versions or standards
Reporting and Action: Communicate findings and improvements
Statistical and Graphical Tools
Control Charts: Identify process stability and variation
Pareto Charts: Show major sources of defects via 80/20 rule
Scatter Diagrams: Correlate code complexity vs defects
Trend Lines: Track quality improvement or decline over time